

Confidential Payroll on a Public Blockchain

A privacy-preserving payroll dApp built with Fully Homomorphic Encryption (Zama FHEVM)

Author: Tornike · Course: Blockchain Technologies · June 2026

Repository: github.com/shubita924/confidential-payroll · Deployed (Sepolia):

0xB0A0A62B1f7e1950096f7eeCEB3991837c1B94f3

1. Overview

On a public blockchain every stored value is world-readable, so a naive on-chain payroll would permanently expose every employee's salary. This project keeps payroll *on* a public chain while keeping the amounts *confidential*, using Fully Homomorphic Encryption (FHE). An employer submits salary payments as encrypted values; the smart contract maintains a per-employee encrypted balance and accumulates payments by performing addition *directly on ciphertext*, never decrypting on-chain. Each employee can decrypt only their own balance, client-side, after proving ownership of their address. The system is deployed and fully functional on the Sepolia testnet, with a browser dApp and automated tests for the core privacy logic.

2. Threat Model — who can see what, who you trust

Data	Employer	Employee (own)	Public observer
Salary amount of a payment	visible	visible	encrypted
Employee's accumulated balance	visible	visible	encrypted
That a payment occurred	visible	visible	visible
Which addresses are on payroll	visible	—	visible

Protected: salary amounts and accumulated balances exist on-chain only as encrypted handles (e.g. `0x1d0fadca...a70400`); inspecting the chain reveals nothing about the numbers. **Not protected (by design):** transaction metadata — since each payment is a public transaction between two addresses, an observer still sees that the employer paid *some* address, and how often, but never the amount. This is the standard confidential-token trade-off.

Trust assumptions. Confidentiality rests on two things: (1) the security of the FHE scheme (lattice-based, believed post-quantum hard), and (2) Zama's threshold key-management network (KMS), which performs a decryption only for an address that the contract has explicitly authorized via its on-chain Access Control List (ACL). No single server or hardware vendor is trusted, and no party ever holds the full decryption key. The employer is trusted only for *input correctness* (entering the right salary); the system protects *confidentiality*, not the honesty of the employer's input.

3. Architecture & Data Flow

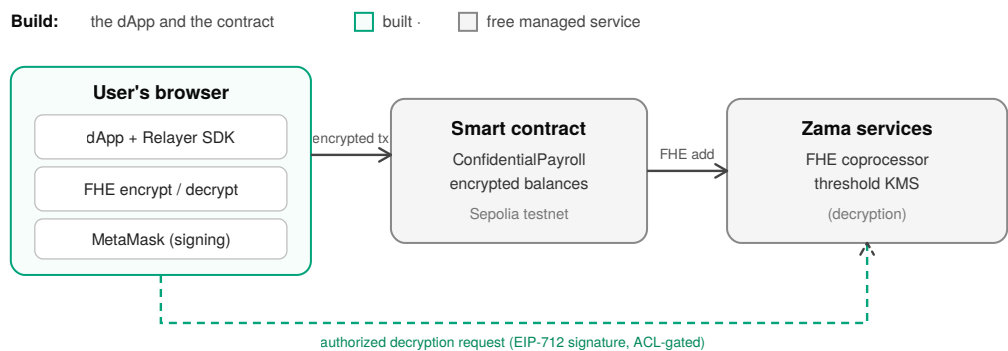


Figure 1 — Components and data flow. Solid arrows: the payment path. Dashed: the decryption path.

Pay flow: the employer types an amount; the browser encrypts it locally (the plaintext never leaves the machine) and sends a `paySalary` transaction carrying the ciphertext plus a zero-knowledge input proof. The contract adds it to the employee's encrypted balance with `FHE.add`; the homomorphic computation itself is executed off-chain by Zama's coprocessor, while the chain stores ciphertext handles. **Decrypt flow:** the employee reads their encrypted balance handle, signs an EIP-712 authorization (no gas), and the threshold KMS returns the plaintext only because the contract previously granted that address access via `FHE.allow`.

4. Cryptographic Choices and Why

The defining requirement is *arithmetic on a secret value, on-chain* — repeatedly adding payments to a running balance. That requirement selects the primitive:

- **FHE (chosen).** Computes directly on ciphertext, single-party, on-chain. It fits the accumulation requirement exactly: the contract updates an encrypted balance without ever seeing it.
- **Zero-knowledge proofs.** Excellent for *proving* statements about hidden data, but the prover must hold the plaintext; there is no shared encrypted state the contract keeps mutating. Wrong shape for an accumulating balance (though complementary — see limitations).
- **MPC.** Would require ≥ 2 non-colluding parties to hold shares of every balance, adding a liveness and collusion assumption and significant operational weight.
- **TEE.** Shifts trust into hardware enclaves — trusting a chip vendor rather than a cryptographic assumption.

The scheme used (TFHE, via Zama) is based on the *Learning With Errors* problem: each ciphertext embeds the value together with small random noise, and recovering the value without the key is computationally hard. Every operation grows this noise; *bootstrapping* periodically resets it by homomorphically evaluating the decryption circuit, enabling unlimited chained operations. This is also why FHE is compute-heavy and why the homomorphic work runs on a dedicated coprocessor rather than on every validator. Access control is enforced cryptographically:

`FHE.allow(handle, addr)` records, on chain, exactly which addresses the KMS may decrypt for — in this project, the employee and the employer.

5. One Failure Mode & Limitations

Metadata leakage (primary failure mode). The scheme hides amounts but not the payment graph: an observer correlating transactions can learn *who* is paid and *how frequently*, even without amounts. Pay-frequency or timing patterns could leak coarse information. Mitigating this would require additional techniques (e.g. stealth addresses or pooled/batched payouts), which are out of scope for this MVP. Two further limitations: balances are *encrypted bookkeeping*, not yet backed by a transferable token (a production version would wrap them in a confidential ERC-20 / ERC-7984 to enable real withdrawals); and the use of `uint32` bounds totals at $\sim 4.29 \times 10^9$, so a production build would use `uint64` with an encrypted bound check via `FHE.select`.

6. Implementation & Verification

Stack: Solidity + `@fhevm/solidity` (encrypted types and operations); Hardhat with the FHEVM plugin for compile, test, and deploy; Sepolia testnet (chainId 11155111); a static HTML/JS frontend using ethers v6 and `@zama-fhe/relayer-sdk` (WASM) with MetaMask. No backend server. **Tests** (mock mode) verify the privacy logic: the deployer becomes the employer; paying 3500 then 1500 yields a decryptable balance of 5000 (proving encrypted accumulation and authorized decryption); and a non-employer call to `paySalary` reverts. Run instructions and the full architecture are documented in the repository README.